

# Notes on Neural Networks: part 3 - CNN

**Alessio Micheli**

[micheli@di.unipi.it](mailto:micheli@di.unipi.it)

*2024*



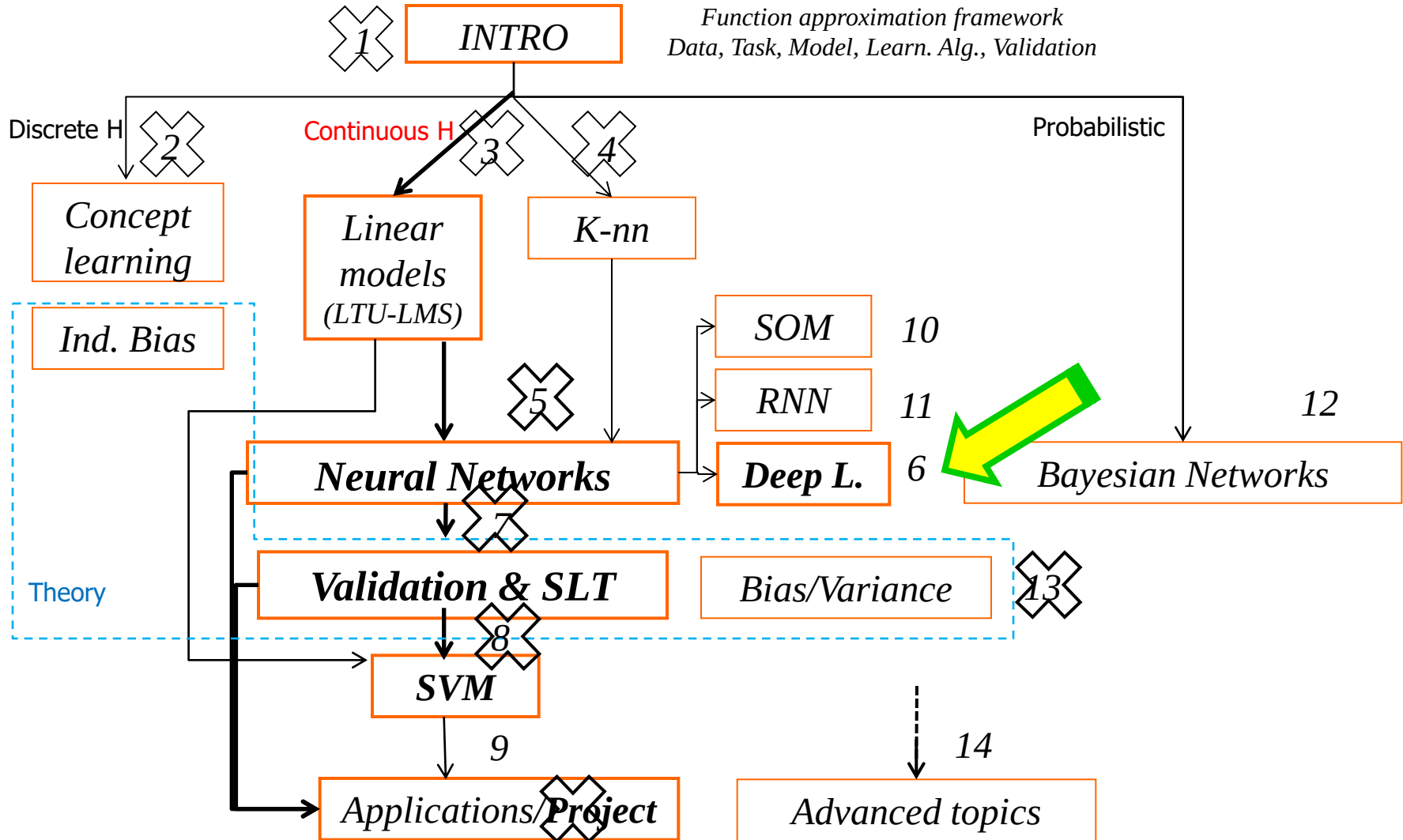
Dipartimento di Informatica  
Università di Pisa - Italy

# ML Course structure

## Where we go



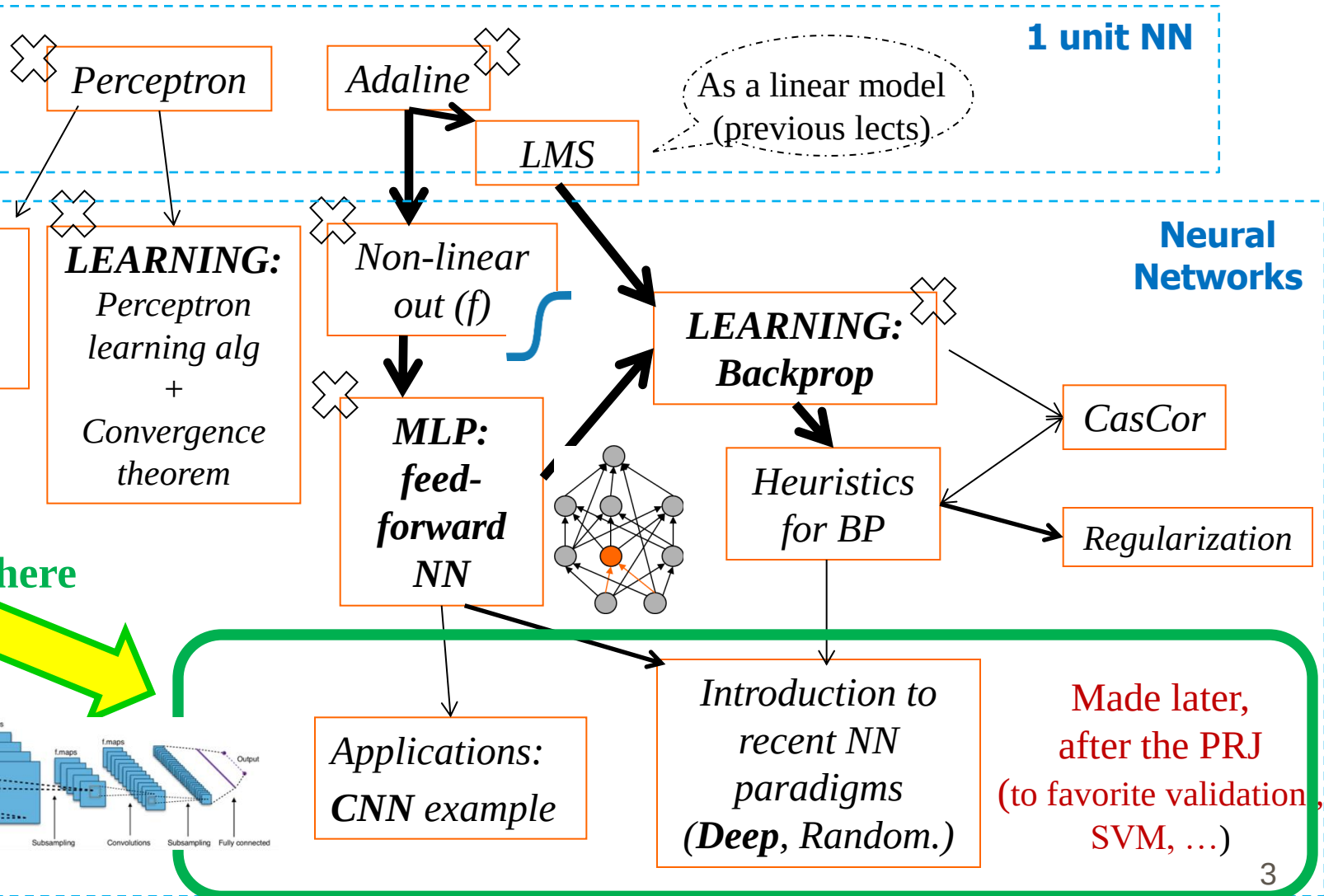
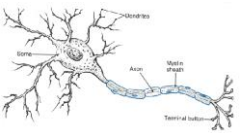
Dip. Informatica  
University of Pisa



# Course structure: zoom in the NNs box



Dip. Informatica  
University of Pisa



# Applications

A huge number of successful applications in all the fields of applications seen for ML (see intro of the course) since his origins

- Performance
- Easy to be configured as a first approach for learning arbitrary functions whose general form is unknown in advance  
(without explicitly assuming a prior statistical models)

Sub-symbolic tasks (especially sensorial data, motor control, visual processing) have been a workhorse of NN.

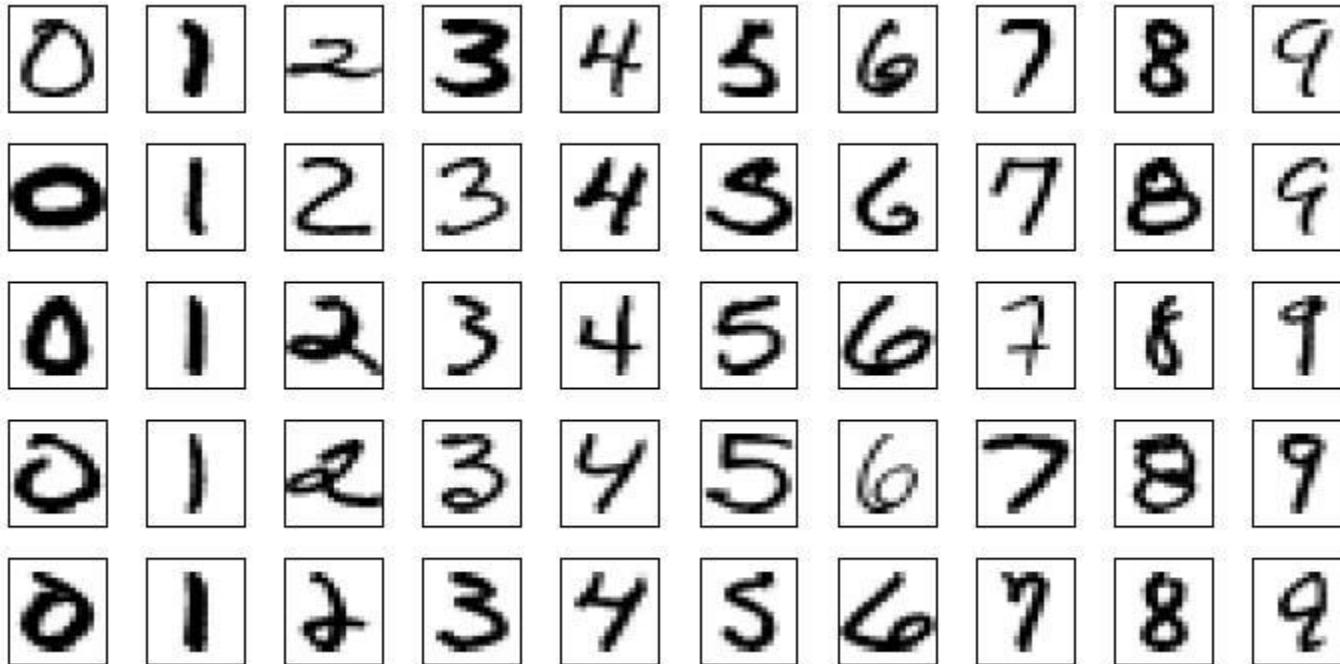
Exploited in robotics, pattern recognition, control, ...

An example:

- Showing the flexibility of NN design
- and historical example on how to **change the design** dealing with specific applications
- A way to implement a not-fully connected FF-NN for a specific problem

see other more standard examples on Mitchell books for face recognition tasks (section 4.7) ... *and nowadays even by newspapers!!!!*

# Character recognition: classification



*Examples of training cases from ZIP code data.*

*Each image is a (normalized)  $16 \times 16$  8-bit grayscale representation of a handwritten digit (Le Cun et al. 1990).*

© Hastie, Tibshirani & Friedman 2009

MNIST benchmark (“the *drosophila*\* of machine learning,”)

# First approaches

- Standard NN with 256 inputs (why?):
  - Misclassification rate of 5%/20% (many errors due to lack of invariance as rotations etc.)
- Basic idea: exploit architecture design to include prior information into NN
  - restricting the network architecture: local connections of receptive fields: extraction of local features
  - constraining weights by *weight-sharing* among different units → reduce the number of free-parameters while allowing more complex connectivity
    - The same operation on different part of the input (image)
    - Because the features of the characters should appear in different parts of the image (regardless the position, small rotations, etc)
  - “LeNet”/CNN/ConvNet: *Convolutional (neural) networks*  
(Refs.: see Haykin or DL book chap. 9)

# The name Convolutional

- The name Convolutional NN takes inspiration from the *Convolutional operator*

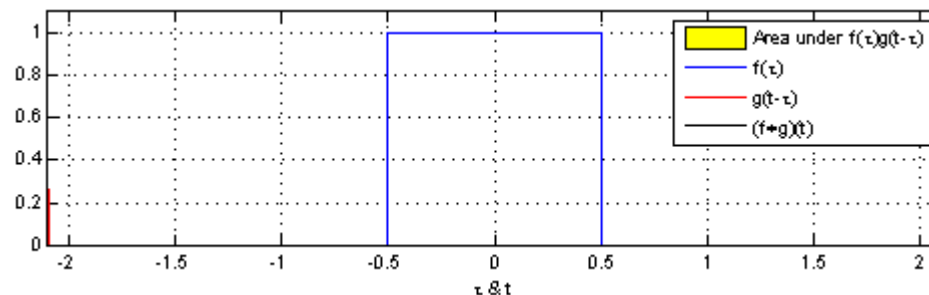
$$\begin{aligned}(f * g)(t) &\stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau \\ &= \int_{-\infty}^{\infty} f(t - \tau)g(\tau) d\tau.\end{aligned}$$

CONCEPT:

Weighted average of a function  $f$ ,  
weighted by another function  $g$   
moved over time (**sliding**)

We are skipping mathematical def. of  
“well defined” convolution etc.

Image  $g$  limited, defined (not zero) on an interval



Animation at

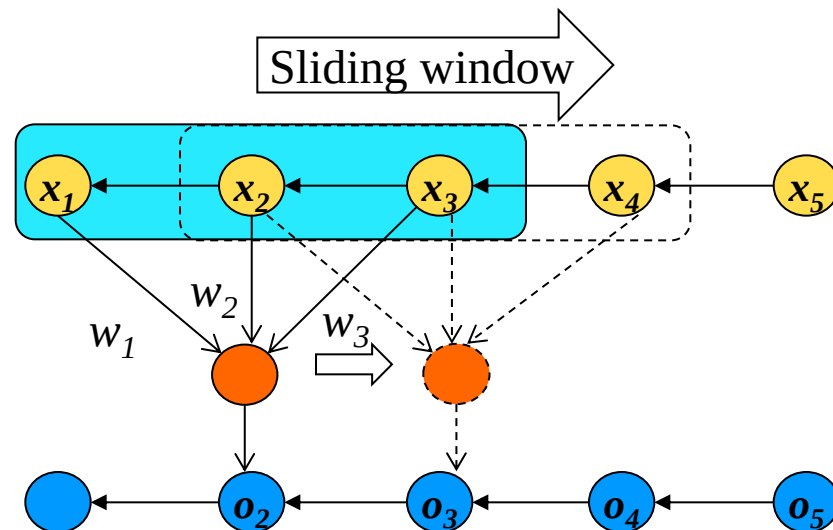
[https://en.wikipedia.org/wiki/Convolution#/media/File:Convolution\\_of\\_box\\_signal\\_with\\_itself2.gif](https://en.wikipedia.org/wiki/Convolution#/media/File:Convolution_of_box_signal_with_itself2.gif)

# CNN details: Convolution

- A simple neural unit example over a stream

$$out_t = \sum_{i=1}^3 w_i x_{t+i-2}$$

**Exercise:** compute  $o_2$  and  $o_3$   
according to the equation above  
(start with  $t=2$ )



Weights are tuned as usual **by learning**:  
An adaptive filter

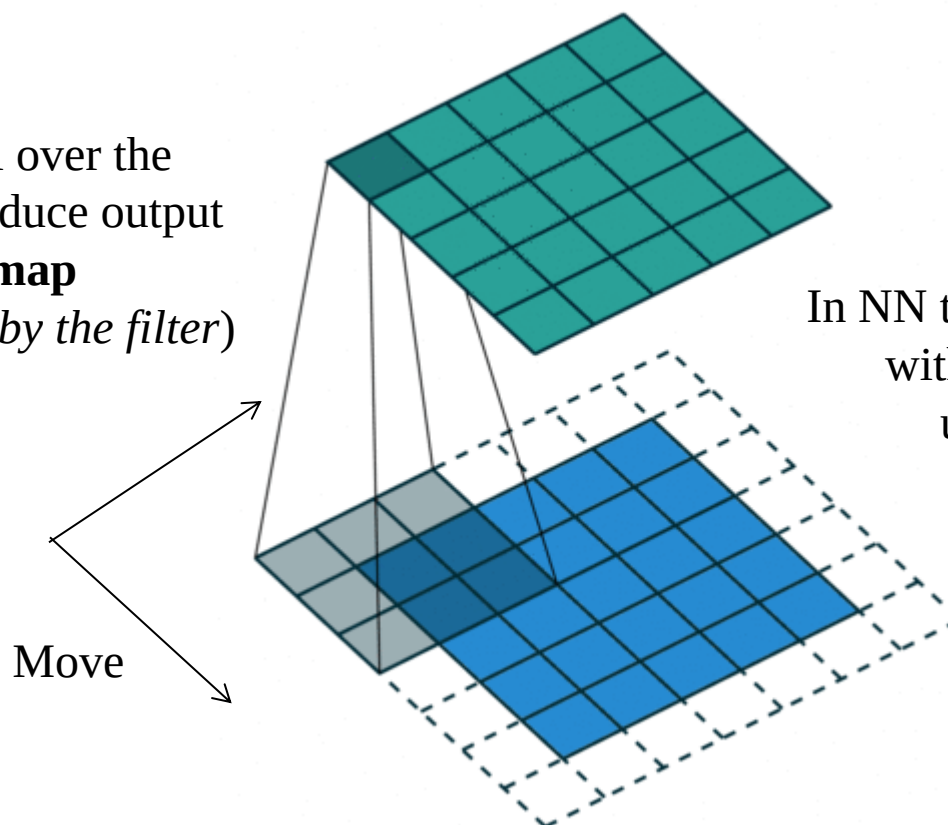
**Weight sharing**  
for the two instances of units  
More units operate as a “filter bank”

We will call this **Time-Delay NN**

# CNN details: 2D Convolution

- 3x3 kernel (unit *local receptive field*), 1 pixel shift (*stride* 1 for traversing)
- Padding is used to treat the border

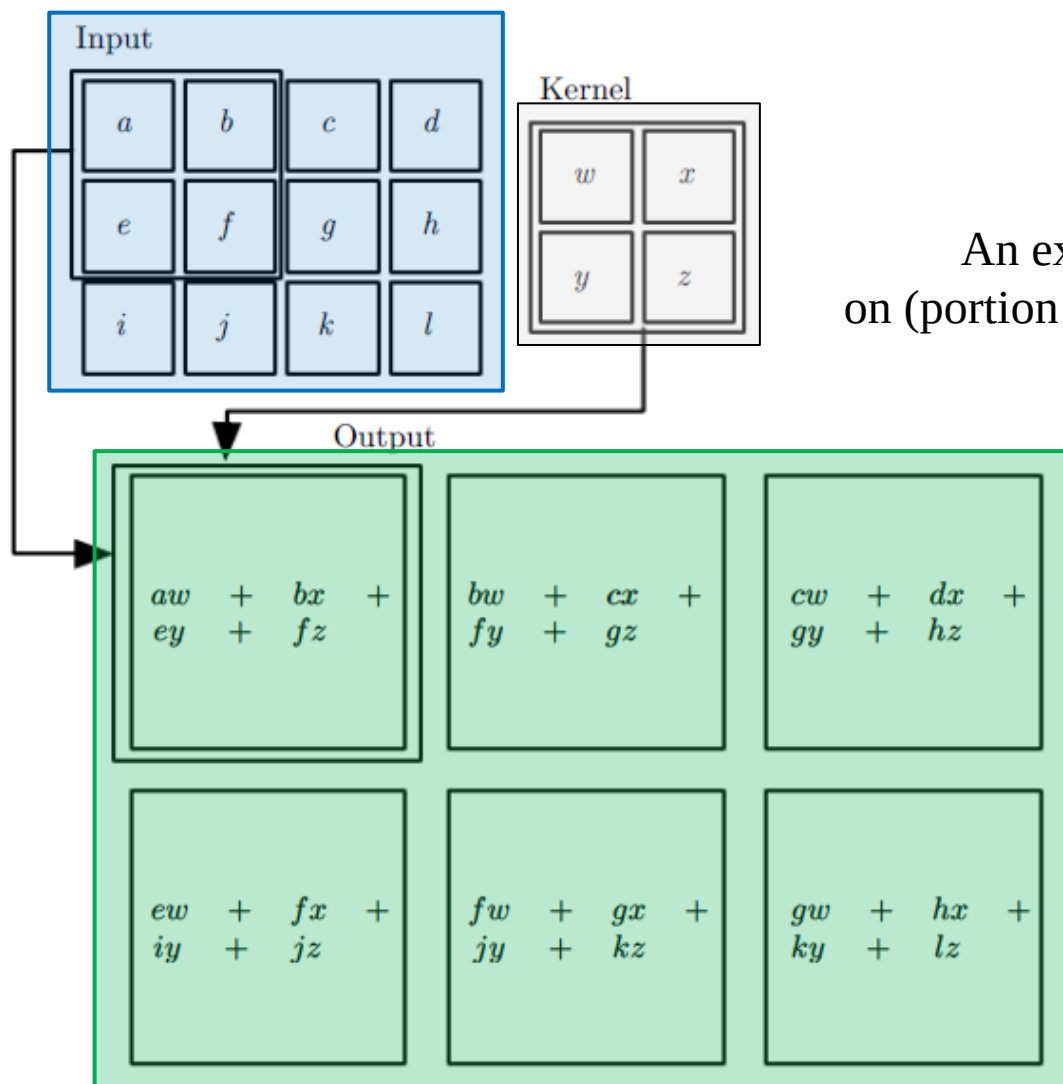
Move the kernel over the  
image (blu) to produce output  
**(feature) map**  
(features extracted by the filter)



In NN terms: we traverse the image  
with (*the same*) neuron that  
uses local connections  
(*receptive field*)



# CNN details: 2D Convolution



An example of 2-D convolution  
on (portion of ) a 2-D (2 dimensional) image

# Convolution in 2D



Dip. Informatica  
University of Pisa

To have a look to an example of implementation over a 2D image  $I$ , with a 2D kernel  $K$

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n).$$

This form is used in many NN library for CNN:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Written as *cross-correlation* form

DL book : 9.1 for details

Not for the oral!!!

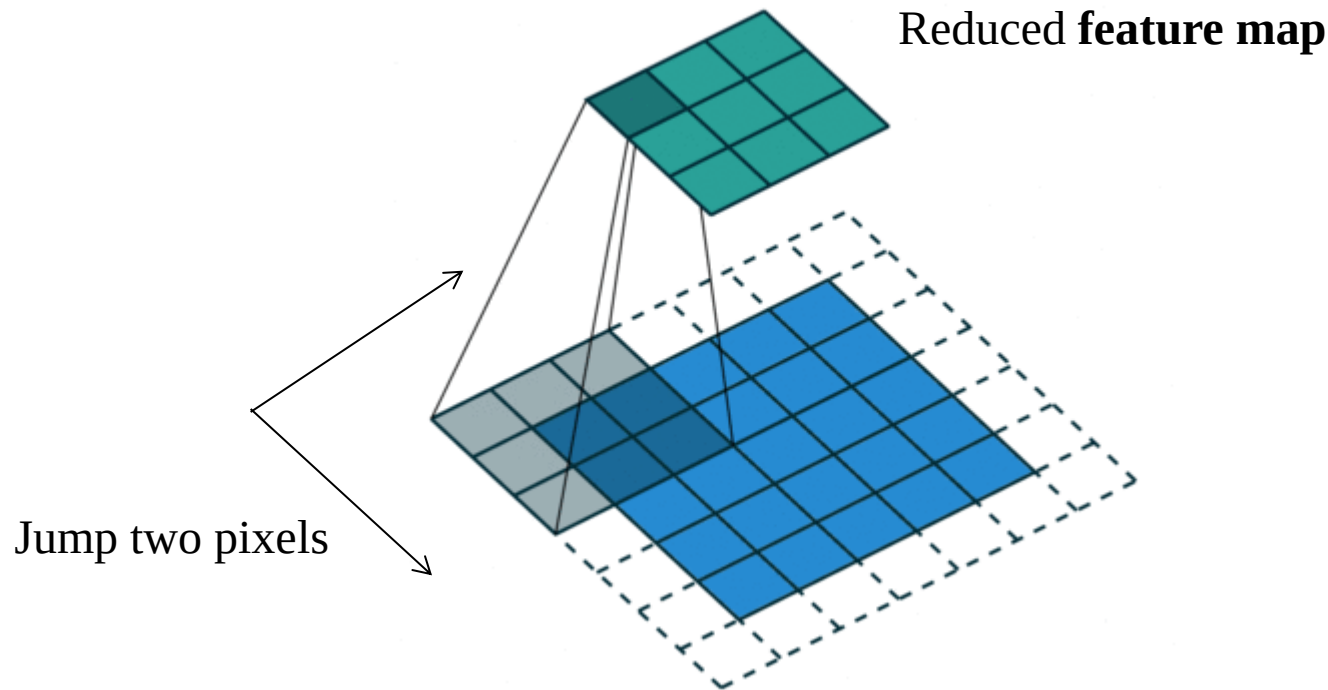
# CNN details: 2D Convolution

## Stride > 1



Dip. Informatica  
University of Pisa

- Down/Sub-sampling: moving jumping two pixels
- Aka: "stride"



# A network of “filters”

The units' weights are a *filter* trained to detect some features/pattern in the image

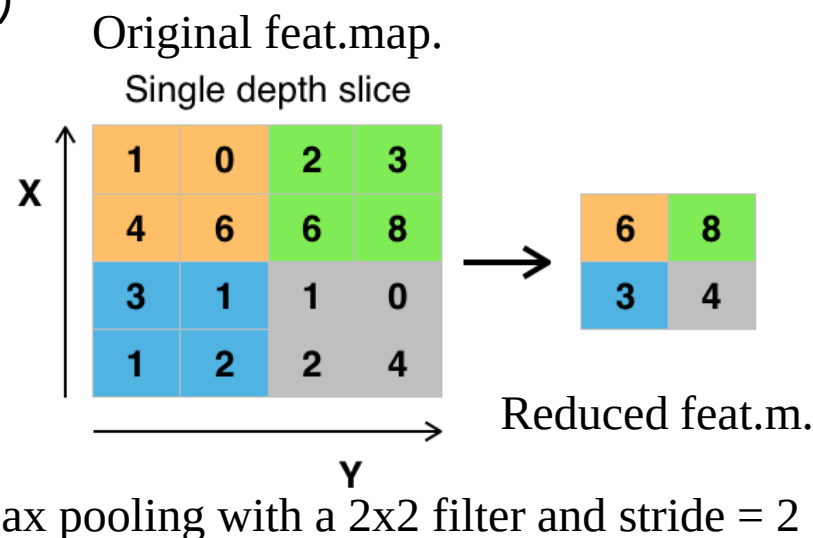
Architecture design

- **Local and small receptive field (kernel):** by this architecture constraints, learnt "filters" produce the strongest response to a spatially local input pattern (similarly to the animal visual cortex).
- The same unit/filter is **applied across the entire image**: allow for features to be detected regardless of their position in the visual field, thus constituting the property of *translation invariance*.
- **Learnable filters**: we can learn the filters that in traditional algorithms for image processing were hand-engineered. This independence from prior knowledge and human effort in feature design is a major advantage.
- **Stacking many layers**: feature maps unit represent larger and larger areas of the original image (assembling areas of the previous feature maps) . See later.

# CNN details - Pooling

- Reduce the feature map by
  - subsampling (see before, stride > 1) or
  - operating a mean (average or weighted average) or
  - **max pool** operation (most common)over the neighbors of each output

In practice instead of producing a value for each input pixel, we produce a value for a set (rectangular) of pixels taking the mean or the max of the kernel/neuron outputs

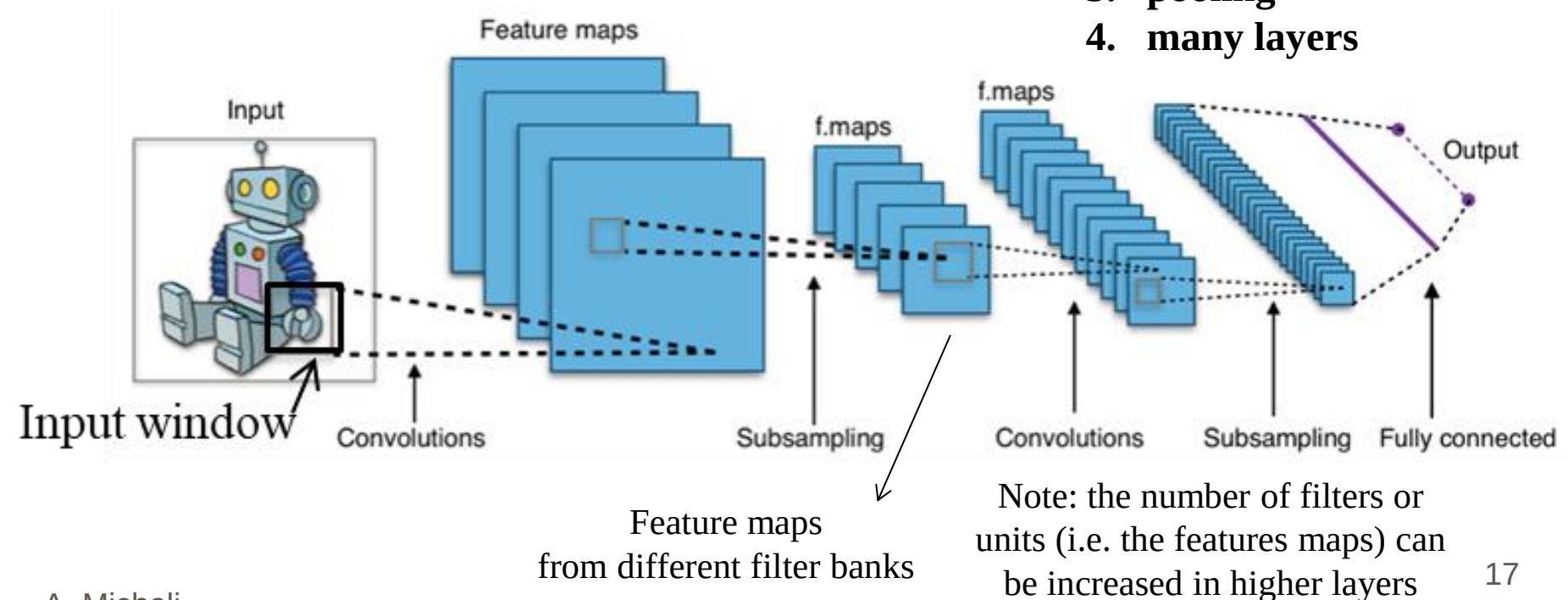


- Pooling also further helps to make the representation become approximately *invariant* to small translations of the input.
  - Because the max/mean has small changes translating few inputs → e.g. a high output is still high for a slight translate version

# CNN as a whole

- CNN exploits weight sharing to set-up a shifting window/local field of the units over a segment of the input signal, also extended to 2D images and re-apply for many layers (feature maps) (activating pooling subsampling/average operators)

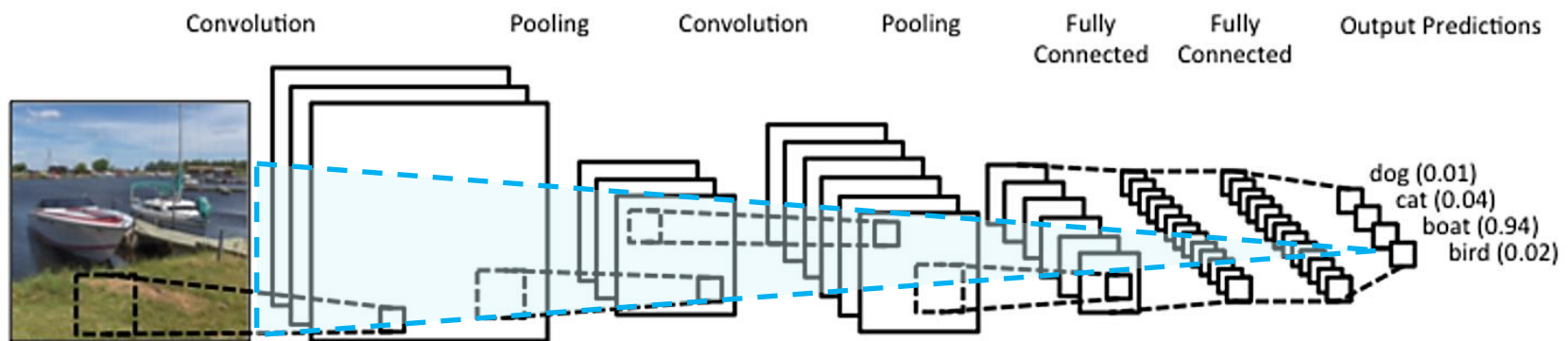
1. **local connections**
2. **shared weights**
3. **pooling**
4. **many layers**



# Second example

- Same number of feature maps after pooling

1. **local connections**
2. **shared weights**
3. **pooling**
4. **many layers**



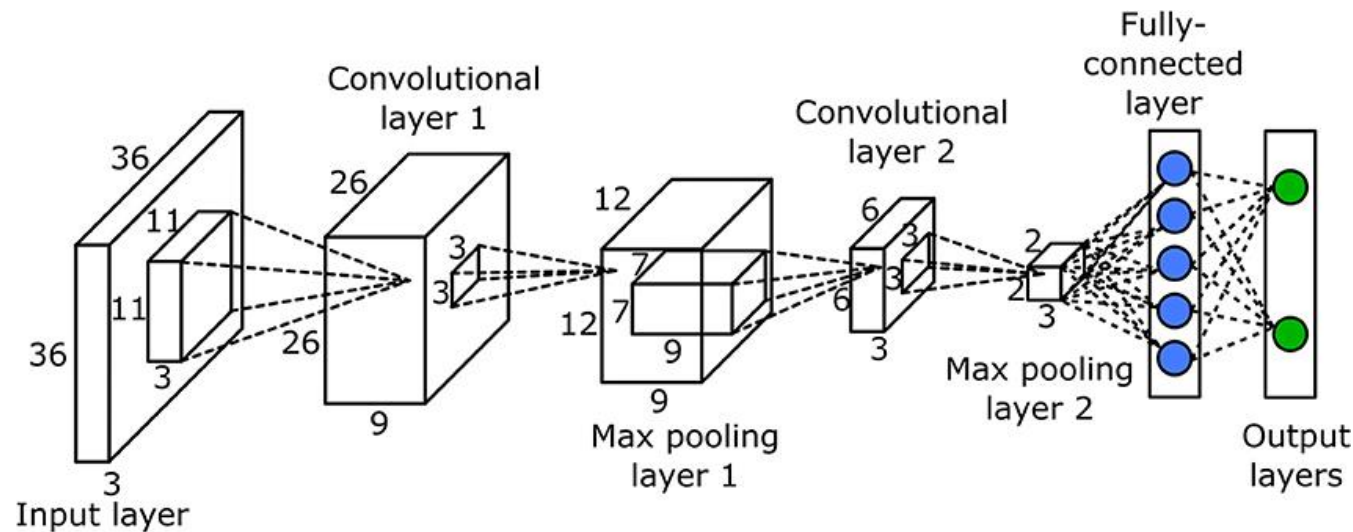
Cone from the image to final maps

(the receptive field of final units is larger than the receptive field of the units in the first layers (we reduced the feature maps many times); they are indirectly connected to large part of the image)

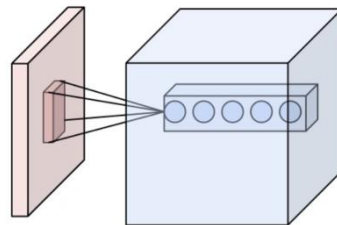
Note: the number of filters or units (i.e. the feature maps) can be increased in higher layers

# Third example

- To be precise (example with numbers and a different view)



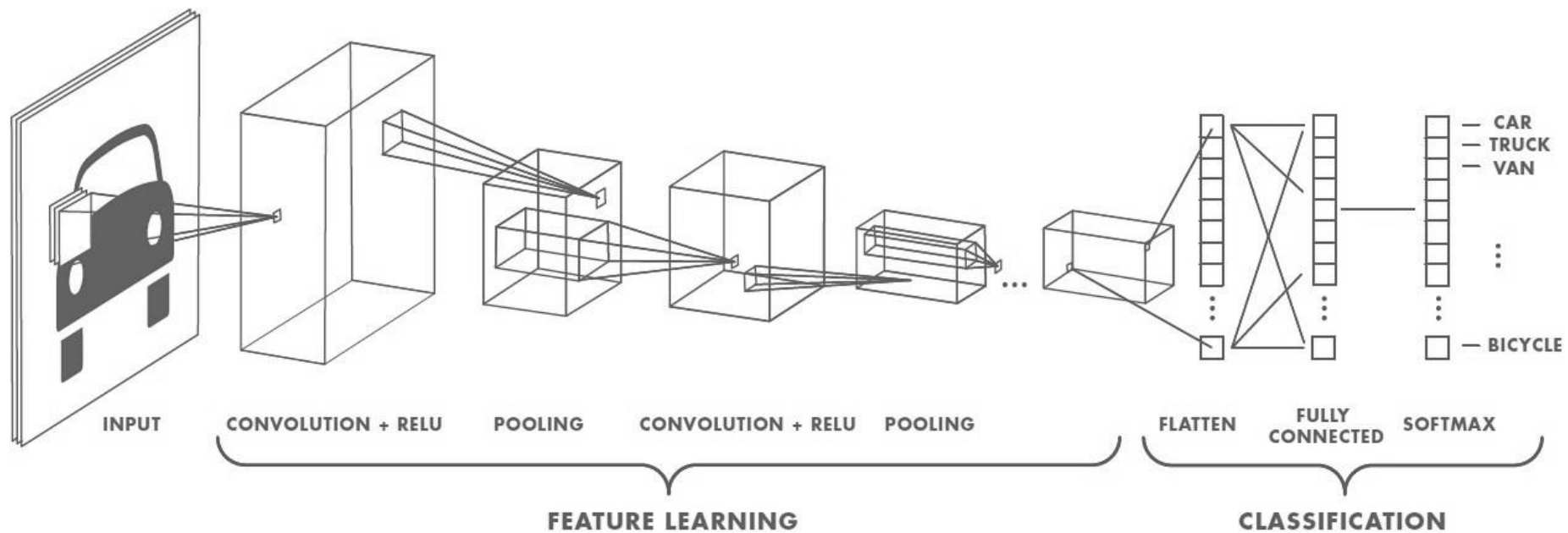
Color image  
RGB



Depth of this volume =  
number of neurons  
(features maps)

# 4-th example

- ~ AlexNet



# Summary of advantages

- Local connections and weight-sharing:
  1. Detect local motifs/patterns in images
  2. and invariant to translation (invariant to location of the pattern)
    - Reduce the number of free-parameters (keeping the connections): regularization!
- Pooling (merge role)
  3. Reduce the dimension of the representation (in each layer → pyramid of layers)
    - Help the invariance to small shift and distortions (that does not have effect of the pool result, e.g. max progress just the hits of a pattern)
- A historical instance of **deep** neural network
  4. 1 & 2 applied to all the features maps\*: progressive abstracting of features from images
    - Making possible the composition of primitives (low-level features, learned one time) for all the possible combinations in the successive layers (see the next DL lectures)
- Visual system (neuroscience) analogies have been studied

1. local connections
2. shared weights
3. pooling
4. many layers

# How to use?

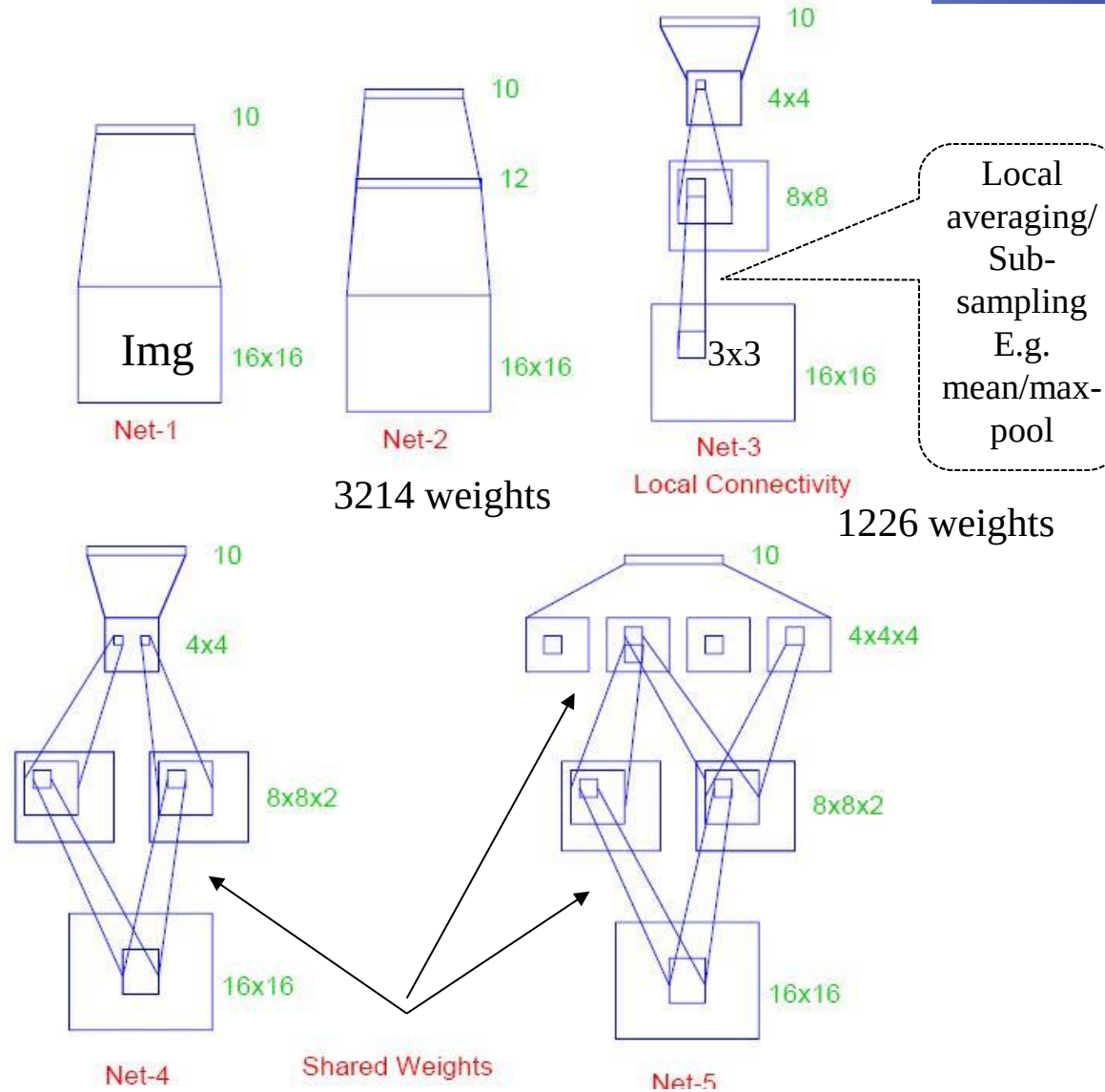
- Training is typically made by *back-propagation*, with the many heuristics that you know (and some specialization of course, e.g. for weight-sharing, the pooling etc.)
- Give the typical usage of **huge networks** and large amount of data, many hyper-parameters are fixed by experience (which unfortunately is not your case yet) or by experts suggestions (!)
  - As it would be very expensive to run cross-validation on large hyper-par. search spaces
- Discussion on
  - the issues of training and
  - on the regularization effects and
  - good behavior of such huge networks, trained with large amount of data, will be discussed later (within deep network framework)

# LeCun architectures also MNIST benchmark



Dip. Informatica  
University of Pisa

No hidden  
layer



*Architecture of  
the five  
networks used in  
the ZIP code  
example.*

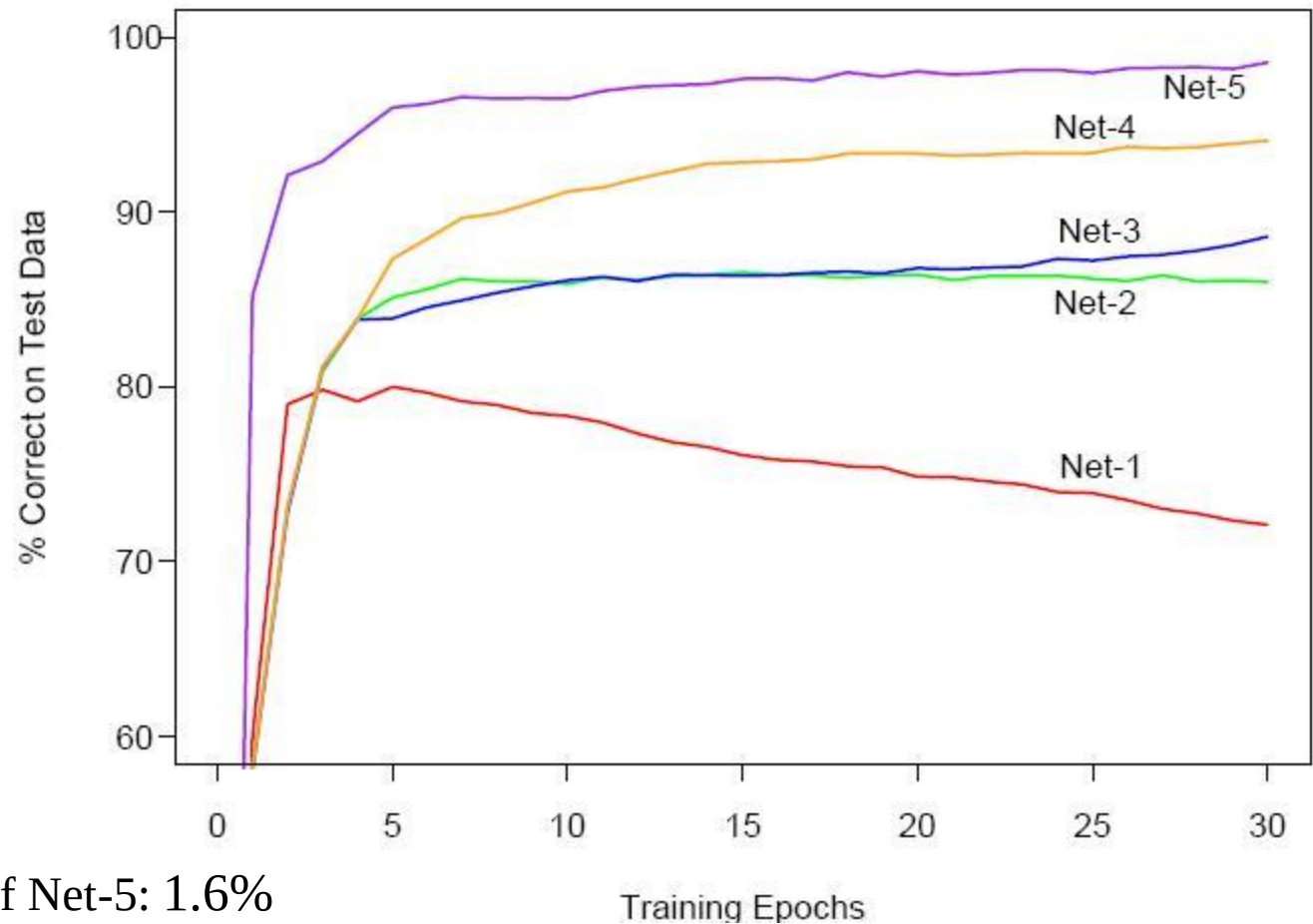
© Hastie, Tibshirani & Friedman 2009

| Net | Conn. | Weights |
|-----|-------|---------|
| 1   | 2570  | 2570    |
| 2   | 3214  | 3214    |
| 3   | 1226  | 1226    |
| 4   | 2266  | 1132    |
| 5   | 5194  | 1060    |

# Test Performances curves

*Test performance curves, as a function of the number of training epochs, for the five networks of Table 11.1 applied to the ZIP code data (Le Cun, 1989)*

*Note: TR error zero for all of them over a reduced set of images (~300)*



Misclassification rate of Net-5: 1.6%

With 5194 connections

1060 weights

Nowadays: up to 0.8% (NN and SVM) and even less...

# MNIST Handwritten Digit Recognition Benchmark



Dip. Informatica  
University of Pisa

- Full data set: 60000 images for training 10000 for test.
- Progress (1998-2012) of the results at <http://yann.lecun.com/exdb/mnist/>
- OR [https://en.wikipedia.org/wiki/MNIST\\_database](https://en.wikipedia.org/wiki/MNIST_database)
- Progress by Deep Learning:
  - E.g. MLP or CNN with many layers
  - Recent methods pre-train each hidden CNN layer one by one in an unsupervised fashion then use supervised learning to achieve **0.39%** (2006)/**0.35%** (2011)
- Best: (2012) **0.23%** error (committee of conv. networks) (2016): **0.21%**, ....Nowadays?
- But (warning!) too many uses of the test data to be still significant (especially for improvements of 0.1% or similar)

# Deep MLP with Backprop on MNIST via GPU



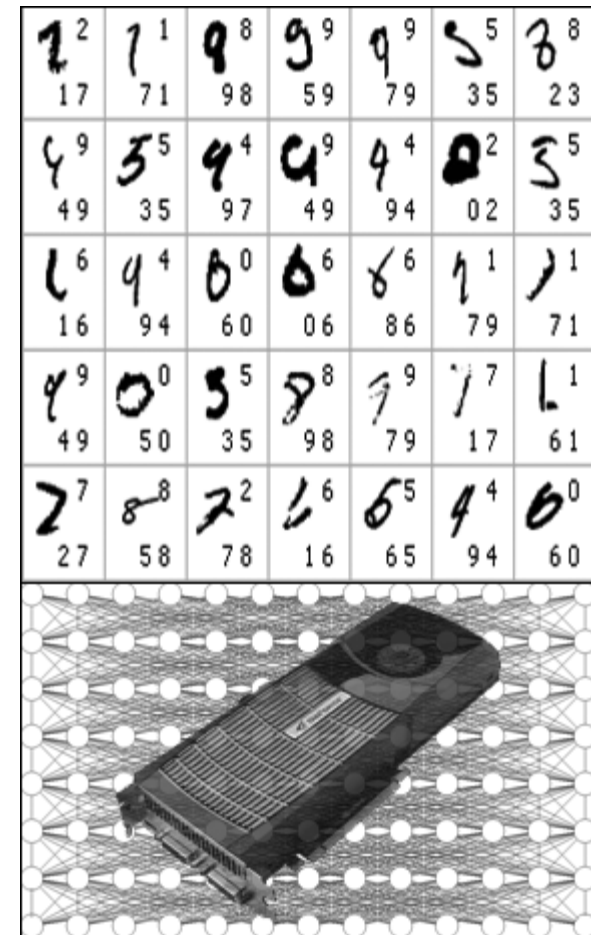
Dip. Informatica  
University of Pisa

But is it really true that deep BP-MLPs do not work at all (gradient vanish, training time etc.), or do they just need more training time?

Paper in Neural Computation, 2010  
<http://arxiv.org/abs/1003.0358>:

*“Good old on-line back-propagation for plain MLP yields a very low **0.35%** error rate on the famous MNIST .... All we need to achieve this best result (as of 2010) are many hidden layers, many neurons per layer, numerous deformed training images, and graphics cards to greatly speed up learning”.*

- 6-layer NN 784-2500-2000-1500-1000-500-10 (on GPU): 12 million weights !!! With 2 hours of training.
- Complexity control issue: deforming images to get much more training instances (\*)



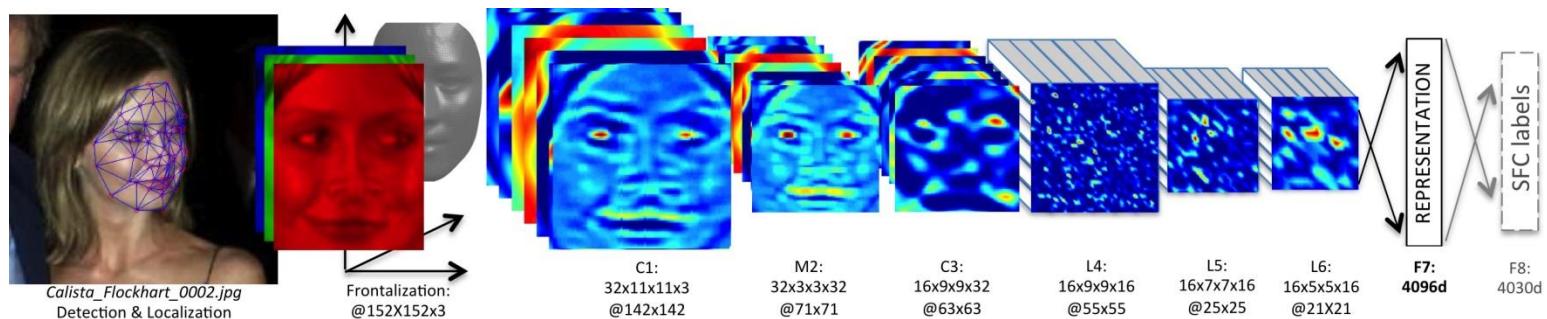
Large MLP and GPU

# An instance on a “no-more recent” result (CVPR, 2014)



Dip. Informatica  
University of Pisa

- **Face recognition** combining (**deep**) **Neural Networks** and other ML approaches. *Note: Convolutional networks* !!! 9 layers → deep.
- Starting from four million facial images belonging to more than 4,000 identities



- Asked whether two photos show the same person, DeepFace answers correctly 97.25% of the time ... just a shade behind humans (97.53%).
- From a news on newspaper LaRepubblica April14 ;-)



# Modern CNN

- Many variants and specialized architecture/models for processing of images → “Intelligent Systems for Pattern Recognition” course
  - High impact in Pattern Recognition and Computer Vision
- Neuroscientific basis of CNN → “CNS” course
- Efficient implementations through tensor representation of matrixes, exploiting GPU (many ML library for DL)
- Even using the features by a models already trained with large benchmarks e.g. from **Alexnet** (winner of the ImageNet object recognition challenge) [Krizhevsky *et al.* NIPS 2012]
- E.g. in Matlab. Also by matconvnet toolbox: googLeNet , vgg, e ResNet \*.
- Many pre-trained CNNs for image classification, segmentation, face recognition, and text detection are available.
- In the following: ImageNet examples + tensors



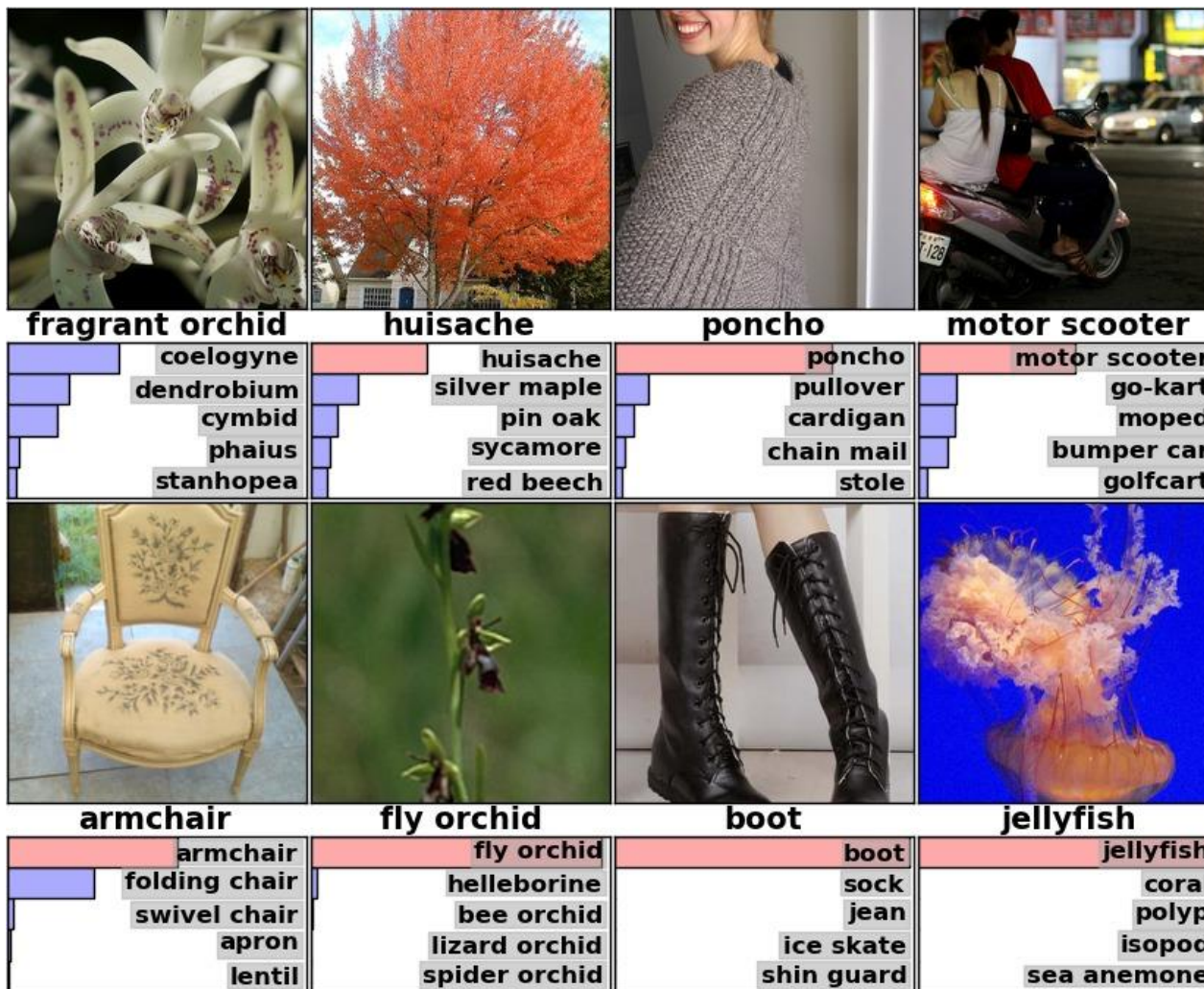
# ImageNet Example

- Krizhevsky, Sutskever\*, Hinton\*\* , *ImageNet Classification with Deep Convolutional Neural Networks*, NIPS 2012
- We trained a large, deep convolutional neural network to classify the **1.3 million** high-resolution images in the ILSVRC-2010 ImageNet training set into the 1000 different classes.
- On the test data, we achieved top-1 and top-5 error rates of 39.7% and (from ~26% to) 18.9% which is considerably *better than the previous state-of-the-art results*.
- The neural network, which has **60 million parameters and 500,000 neurons**, consists of **five convolutional layers**, some of which are followed by max-pooling layers, and two globally connected layers with a final 1000-way softmax.
- To make training faster, we used *non-saturating* neurons and a very efficient **GPU implementation** of convolutional nets.
- To reduce overfitting in the globally connected layers we employed a new **regularization** method that proved to be very effective (*dropout*).

\* Sutskever: Chief Scientist of OpenAI

\*\* Hinton: Turing award 2019

# Results demo



Top-5 predictions for random images from the test set

# How to parallelize linear operations on GPU



Dip. Informatica  
University of Pisa

Matrix multiplication (i.e., dot product) is a typical linear operation performed efficiently on GPU

$$(AB)_{ij} = \sum_{k=1}^m A_{ik} B_{kj}$$

Parallelizing the sum  
or a set of independent products (for each  $i,j$ )

$$A_{ik} B_{kj}$$

**A** Input matrix

Many kernel operations

$$W_1, W_2, \dots, W_5$$

We can build **by hand** a block matrix in order to perform many kernel operations as a matrix multiplication on GPU

$$\begin{bmatrix} W_1 \\ W_2 \\ W_3 \\ W_4 \\ W_5 \end{bmatrix} A = \begin{bmatrix} W_1 A \\ W_2 A \\ W_3 A \\ W_4 A \\ W_5 A \end{bmatrix}$$

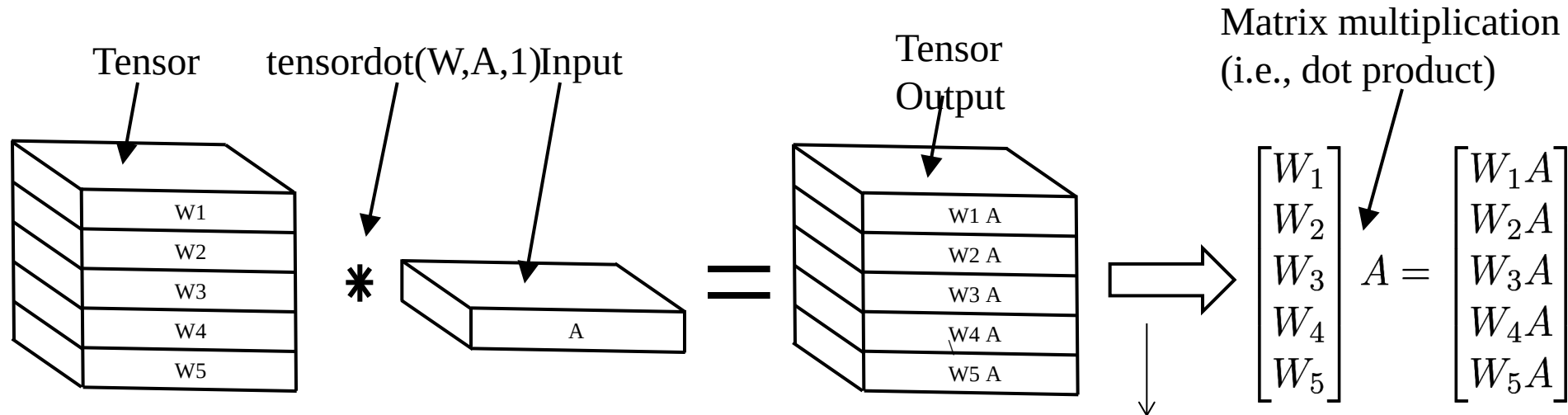
# Tensor Notation for GPU



Dip. Informatica  
University of Pisa

In computer programming a **Tensor** is a multi-dimensional matrix with a linear operation called **tensordot**

**Solution!:** In **Tensorflow**, or other libraries (e.g. NumPy), we can perform many linear operations (i.e., dot products) as a **tensordot** between **Tensors**

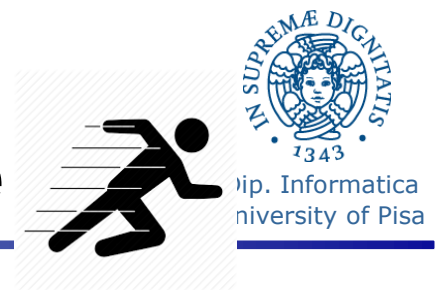


## Why use Tensor notation in Machine Learning?

The use of **Tensor Notation** allows to parallelize many linear operations (i.e., matrix multiplication) between **tensors** on GPUs without construct complex block matrices by hand

“Correspond to”  
but more useful  
if written as  $*$

# Beyond CNN: an example



## *Capsule (Neural) Networks*, [CapsNet] November 2017

- S. Sabour, N. Frosst, G. E Hinton, Dynamic Routing Between Capsules  
<https://arxiv.org/abs/1710.09829>
- Using specialize subnetworks for image processing
  - A "capsule" is a subset of neurons within a layer that outputs both an "instantiation parameter" indicating whether an entity is present within a limited domain and a vector of "pose parameters" specifying the pose of the entity relative to a canonical version. [position and orientation]
  - *State-of-the-art\** performance on MNIST and is considerably better than a convolutional net at recognizing highly overlapping digits.
  - Most interestingly, without transforming image preprocessing
- Routing mechanism (instead of Max-pooling)
  - The brain must have a mechanism for "routing" low level visual information to what it believes is the best capsule for handling it.

# Other architectures (guide slide)



Dip. Informatica  
University of Pisa

## ***Introduction to recent NN paradigms***

- Next lectures: Other architectures.....
  - Deep learning
  - Random weights NN

AKA: *Deep* learning and *extreme* learning ;-)

# Bibliography (NN third part)

---

Please follow the many references trough the slides

**For CNN see Haykin or DL book chap. 9**

DL book:

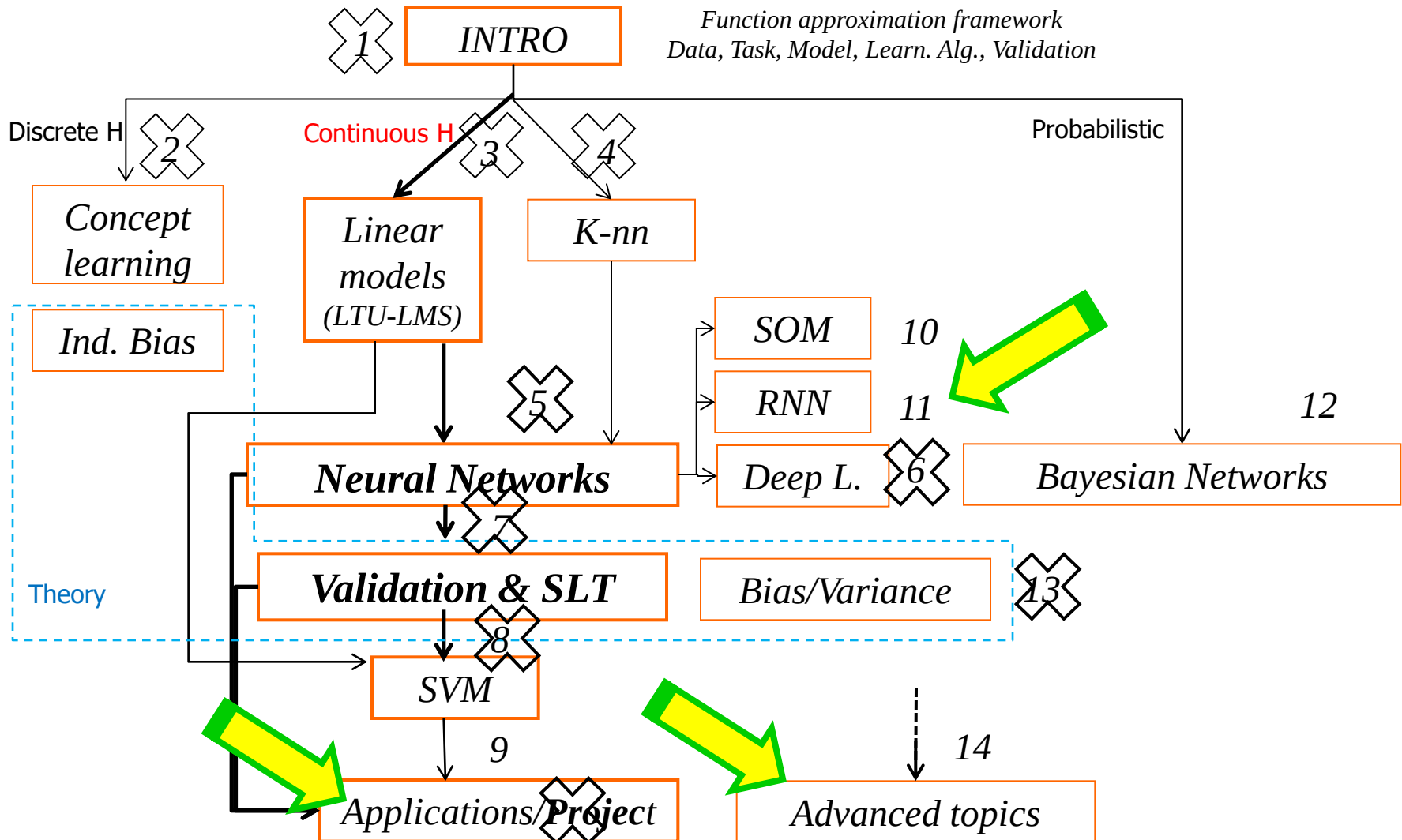
Goodfellow, Y. Bengio, A. Courville: ***Deep Learning***,  
MIT Press, 2016 . On-line: <http://www.deeplearningbook.org/>

# ML Course structure

## Where we go



Dip. Informatica  
University of Pisa



# Notes on Neural Networks: part 3 - CNN

**the end**

**Alessio Micheli**

[micheli@di.unipi.it](mailto:micheli@di.unipi.it)

*2024*



Dipartimento di Informatica  
Università di Pisa - Italy